FYP Project Plan

Adaptive Replanning for Manager Agents under Unexpected Changes

By: Rachel Tan Min Zhi

Research Direction

- Adaptive Replanner baseline: can be like Graph-CoT
- While the benchmarks for adaptive replanning: can be like GRBench
- But importantly we have to go in-depth into adaptive replanning
- Can consider potential implementation in ergon
- Lit Review page: I should think about what people are doing out there for these tasks
    - Consider agentic benchmark designs

Definition

Adaptive Replanning:

- Dynamic process where an AI agent evaluates execution results in real time
- Detects errors or unexpected changes
- Rewrites its remaining steps to achieve the goal
- Unlike rigid, scripted pipelines, it enables resilient, trial-and-error problem-solving

Concerns

- Main considerations I think I should consider
    - Whether it's a meaningful problem
    - Whether there's a research gap
    - The framing
- **Demand unclear:** I don't know if people need a manager agent solution for this kinda problem
    - **For the problem:** AWS's Well-Architected Agentic AI Lens (AGENTREL04) lists hard-coded orchestration that can't adapt when agents become unavailable as a common issue, and recommends fault-injection testing of fallback chains.
    - **Against needing a manager:** the same AWS document recommends deterministic capability-based routing and ordered fallback chains, so rules may suffice where a substitute exists.
    - I found no data on demand for manager-style solutions.
- **Perhaps it's more useful if no fallback exists** → and it would justify a manager agent approach
    - No evidence as to how often this happens in real systems
- **Benchmark would address this uncertainty**
    - **Research question:** under unexpected external team changes, when do rule-based approaches suffice, and when is a manager or replanning policy needed?
    - It compares rule-based and manager approaches with task goals and requirements held fixed.

    - **Out of scope for now:** changing requirements (task and preference drift), kept as future work.
    - Either outcome is a finding: if rules suffice, builders can avoid the cost of a manager; if a manager wins, the benchmark locates where.
- **Answering: exactly what kind of problem needs a manager agent?**
- **So essentially let's turn uncertainty into a research question**

Problem

Title: Adaptive Replanning for Manager Agents under External Team Changes

- We study the manager agent's role in resilient multi-agent workflows
    - How it should repair a plan after an unexpected agent loss
    - Evaluated against rule-based and prompt-only baselines on a benchmark whose change processes come from public incident and release records.

Summary

- **1st contribution:** a grounded benchmark for testing disturbances, starting with team-based change. Existing problem: many benchmarks use scripted or invented disruptions that are not derived from real change records.

- **2nd potential contribution:** an explicit continue/repair/replan policy for a manager agent handling external team changes. Task and preference drift are future work.

- **To my knowledge, no existing work combines:**
    - adaptive replanning
    - unexpected external agent unavailability
    - a manager that delegates to specialised sub-agents.

FYP Problem Statement:

Existing agent benchmarks increasingly draw on real-world tasks, such as real user requests (VitaBench; He et al., 2025) and real software issues (SWE-bench; Jimenez et al., 2024). REALM-Bench (Geng & Chang, 2025) evaluates multi-agent planning under scripted environmental disruptions, and MA-Gym (Masters et al., 2025) includes scripted worker joins and leaves in manager-agent workflows.

However, to our knowledge, no existing benchmark isolates how well a manager agent recovers a dependency-rich workflow when agents become unexpectedly unavailable, such as through outages, rate limits, or deprecations. MA-Gym scores whole-workflow outcomes, where a manager that plans upfront can score well without adapting, and REALM-Bench's disruptions concern environments and resources rather than agents. Moreover, to our knowledge, in both, disruption timing and types are authored by the designers rather than derived from empirical records of real agent and tool failures. We address these gaps with a benchmark that gates

tasks on the change, measures recovery directly, and fits its outage and change processes from public incident and release records.

What we Test

Question. Under unexpected external team changes (an agent becomes unavailable), when do rule-based approaches suffice, and when is a manager or replanning policy needed?

Sub-questions

1. Does **telling the manager about the change** in its prompt suffice, or does an explicit continue/repair/replan **policy** improve recovery? (This answers your open "beyond passing the change into the prompt" question.)
    a. **Plain manager:** an LLM manager that is just told in its prompt what changed and left to work it out.
    b. **Policy manager:** a manager with an explicit rule for deciding whether to continue, repair, or replan, based on which tasks depended on the lost agent.
    c. **Why it matters:** if a manager beats rules, you need to know whether that's only because it's an LLM, or because of your policy. Otherwise a reviewer asks what the policy adds. This is your own open question, "is there a point beyond just passing the change into the manager agent's prompt".
2. How do results change with the share of affected tasks that have an equivalent remaining agent (none to all)? [KIV]
    a. Say the finance agent goes down. Some tasks it was handling could be done equally well by another remaining agent, and others could not.
    b. The share is the fraction of affected tasks that have such a substitute. At 100% a simple fallback should work. At 0% there is nothing to route to.
    c. You'd run scenarios across that range and plot recovery against the share, to see where rules stop being enough.
3. At what cost (tokens, time, unnecessary reassignments)?
    a. Manager that recovers slightly better but uses much more tokens may not be worth it

Fixed as of now: the task goals and requirements, and accurately declared agent capabilities.

Varied: change type and duration (outage, rate limit, permanent deprecation)

- sampled from public records; substitute availability
- what the manager is told (unannounced signals such as timeouts, errors, and degraded outputs, with an informed condition as an upper bound)
- workflow size and dependency depth as parameters.

Systems under test:

- Do nothing (supposedly assigned to a human)

- Rule-based router with ordered fallbacks
- MA-Gym's Random, CoT, and Assign-All manager
- Prompt-informed manager
- Policy-based manager

Measures:

- post-change score on affected tasks (deterministic checklist)
- disruption cost (unnecessary reassignments, with unaffected tasks as a control)
- tokens and time, and partial completion.

Out of scope: error recovery, task and preference drift, changing requirements, joins, and deployment-performance claims. Either outcome (rules suffice or a manager wins) is a finding.

Main considerations for benchmark

- Grounded benchmark
    - Grounded in public records of real deployments

- Specialised agents
    - Think real-world solutions have both generic and specialised agent
        - Evidence is a bit weak for whether specialised agents are used in deployments (could not find a lot of reliable sources
        - specialised agents are realistic and widely advocated, but I can't prove it is the dominant real-world pattern
    - But it's ok since the priority is on the benchmark, we will evaluate different kinds of systems on it
        - Rule-based baseline
        - Manager agent
    - Conclusion:
        - Specialised multi-agent systems are realistic but their prevalence in production is unproven
        - So ground benchmark in documented deployments and treat agent roles and team-membership changes as explicit assumptions rather than universal industry practice.

- Long horizon? not necessarily requireed
    - Some sources say deployed agents only run around 10 steps
    - Ground typical lengths in documented use cases where possible
    - Or make it a parameter
    - If we choose to work on long horizon problems → explicitly evaluate dependency accumulation and recovery over extended execution.

- Is there a point beyond just passing the change into the manager agent's prompt

- Make sure the benchmark really treats these as unannounced. Otherwise a reviewer will ask how it differs from MA-Gym's scripted leaves.
    - The changes should be unannounced and rightfully so
    - Not like intentionally announced
    - Tool acting differently / agent time out

Purpose

- **Researchers:** have no good way to compare manager and replanning solutions on this problem.
- **Developers:** could benefit if the benchmark tests realistic disruptions their evaluations miss and gives evidence about which systems recover reliably. A log-import mode is a later stretch.

- Current evaluations of adaptive orchestration largely test change using **generic errors or scripted disruptions that are not grounded in real change records**.
    - Flow (Niu et al., 2025) simulates failures by masking subtask outputs, on three tasks with five trials each.
    - OrchMAS (Feng et al., 2026) tests replanning on standard QA, math, and summarization datasets.
    - SagaLLM (Chang & Geng, 2025) evaluates four scripted scenarios (such as a flight delay or traffic alert at a fixed time) drawn from REALM-Bench.
    - Benchmarks designed for disruption likewise script their change processes: REALM-Bench (Geng & Chang, 2025) and MA-Gym (Masters et al., 2025)
    - Add MCPEvol-Bench derives its tool-change mutations from real MCP server histories → grounded in real records [example to refer to]
        - Covers tool evolution for single agents not a manager handling team change.

- **Without an explicit continue/repair/replan policy, a manager either stalls on the change or restarts work it didn't need to.**
    - **What's lacking:** Existing managers lack an explicit continue/repair/replan policy: in MA-Gym, LLM managers struggle to balance goal completion, constraints, and runtime, and a reactive CoT manager underperforms an upfront planner on goal completion (Masters et al., 2025).
    - **Hypothesis:** after a change, such managers over-react (restarting unaffected work) or under-react (stalling). The right decision is structural, since what to keep, repair, or redo depends on which tasks depend on the lost agent.

    - **Supporting evidence:** Scoped repair preserves valid parts of a plan in related settings: assignment-preserving replanning reached the lowest reassignment rate (17.7%) in multi-UAV allocation, at the cost of higher mission cost and lower feasibility than global replanning (Hwang et al., 2026), and task-decoupled planning cut token use by up to 82% in LLM long-horizon tasks (Li et al., 2026).

- **[For specialised agents] Simple fallbacks fail when agents aren't interchangeable.**
    - Where a close substitute exists, fallback chains and capability-based routing (cite) suffice, since they assume an equivalent agent is available.
    - When agents are not interchangeable, they do not: in Adaptive Orchestration, routing to non-specialist generalist agents lowered task success from 41.6% to 24.3% (Lukei & Kowol, 2026). The manager must then re-plan around what the remaining agents can do.
    - We vary the share of agents with a substitute, so results show where rules suffice and where re-planning is needed.

Clarifications

- **It's not about error fixing.** Error recovery is about wrong outputs and is a crowded area. Here nothing is wrong with the work, and what's lost is capacity.

Current scope

- **In:** unexpected external unavailability (outages, rate limits) of agents whose capabilities aren't interchangeable. The manager must repair only the affected parts of the plan with minimal disruption to the rest.
- **Method:** an explicit replanning policy tied to the dependency structure.
- **Evaluation:** on MA-Gym against its three baseline managers, with new scenarios and metrics. MA-Gym supports joins and leaves, but its workflows and workflow-level scoring don't isolate replanning. Metrics could be post-change score on affected tasks and disruption cost (unnecessary reassignments, with unaffected tasks as a control).
- **Out for now:** error and failure recovery, and preference and task shifts.

Future scope

- Other unexpected changes (joins, task and preference drift)
- Keep the benchmark extensible so new disruption types can be added.

Future considerations

- Will benchmark grounded in real change / manager agent with replanning policy be the selling point

Dev Work

1. **Research grounded-benchmark construction.**
    a. SWE-bench
    b. VitaBench
    c. MCPEvol-Bench
    d. Turn real data into test cases.
2. **Design the change-event schema first.**
    a. E.g.,: timestamp, which agent, event type, duration, whether it's announced.
    b. This one schema serves both public records and developers' own logs, so the log-import idea becomes a later "replay mode", not a separate build.
3. **Build a minimal benchmark:**
    a. Gated tasks
    b. Deterministic checklists
    c. Counterfactual runs: same task, workflow, and seed executed twice, once with the change and once without
    d. Include the rule-based, CoT, baselines.
4. **If handling different changes**
    a. Ensure extensible
5. **Add logging and explainability**
    a. **Dev implementation: log-import:** letting developers feed in their own system logs (records of agent outages, rate limits, releases) so the tool turns them into change scenarios and tests their manager on them.

Notes on your points

- **Easy to integrate:** a small fixed interface (what the system observes, what actions it can take) plus a config for the change process, with seeds. Write the "user's perspective" as a quickstart example before the code.
- **Deterministic by construction:** LLM worker calls add noise, so offer a deterministic worker mode for the core tests.
- **Explainability:** start with a structured trace of change events, manager decisions, affected tasks, and the reason each task passed or failed. That also checks your own benchmark's correctness. Skip dashboards.
- **Log import:** keep it a stretch. It's the most scope and the weakest link to your "grounded in public records" claim.
- **"Mark it multi-agent":** what do you mean? If you mean tagging it as a multi-agent benchmark in the paper and repo, that's just labelling. If you mean a design requirement, say which.

References

- **Hwang et al.:** Se-yun Hwang et al. (2026, *Applied Sciences*), Assignment-Preserving Replanning for Dynamic Task Allocation in Multi-UAV Systems.
- **Li et al.:** Yunfan Li et al. (2026, arXiv), Beyond Entangled Planning: Task-Decoupled Planning for Long-Horizon Agents.